
Sorting Algorithms

Introduction

Sorting Problem

Sorting Problem

Given a sequence $A = \langle a_1, \dots, a_n \rangle$, find a permutation $\langle a'_1, \dots, a'_n \rangle$ of A such that $a'_1 \leq \dots \leq a'_n$.

Key-Value Pairs

Key-Value Pair

- ▶ Two linked data items
- ▶ Key: (often unique) identifier
- ▶ Value: data associated with the key

Examples

- ▶ Student-ID – student name
- ▶ property name – property value
- ▶ object – counter

In Data Structures

- ▶ Key is used to store and find data.
- ▶ Values is mostly ignored.
Most of the time, we will only talk about managing the keys.
- ▶ Key and value are stored together.

Stable Sorting

Stable Sorting

A sorting algorithm is *stable*, if the relative ordering of equal elements is preserved.

5	2 _a	4	6 _a	3	1	6 _b	2 _b
---	----------------	---	----------------	---	---	----------------	----------------

Input

1	2 _a	2 _b	3	4	5	6 _a	6 _b
---	----------------	----------------	---	---	---	----------------	----------------

Stable

1	2 _b	2 _a	3	4	5	6 _a	6 _b
---	----------------	----------------	---	---	---	----------------	----------------

Unstable

In-Place Sorting

A sorting algorithm is *in-place*, if it requires only $O(1)$ additional memory.

Notes

- ▶ Sometimes $O(\log n)$ is also accepted as in-place.
- ▶ Counting requires $O(\log n)$ space.
- ▶ In this class: A word ($O(\log n)$ bits) requires $O(1)$ space.

Recursive Calls

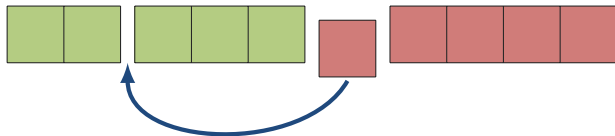
- ▶ Calling a function requires space!
- ▶ Local variables and return address are stored on the system stack.

Insertion Sort

Insertion Sort

Idea

- ▶ Sorted and unsorted part of data.
- ▶ Pick an element from the unsorted part.
- ▶ Insert it at the correct place in the sorted part.



Insertion Sort

Input: An array A with size n .

```
1 For  $i = 1$  To  $n - 1$ 
2   Set  $j := i - 1$ 
3   While  $j \geq 0$  and  $A[j] > A[j + 1]$ 
4     Swap  $A[j]$  and  $A[j + 1]$ .
5      $j := j - 1$ 
```

Properties

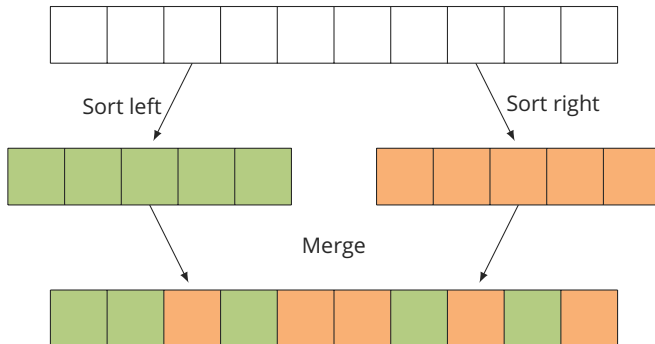
- ▶ In-place, Stable
- ▶ $O(n^2)$ comparisons, $O(n^2)$ swaps
- ▶ Close to linear if nearly completely sorted.

Merge Sort

Merge Sort

Idea

- ▶ Split data in two parts.
- ▶ Sort each part.
- ▶ Merge sorted parts together.



Merge Sort

Input: An array A of size n and two indices $start$ and end with $0 \leq start < end \leq n$.

1 **Procedure** *MergeSort* ($A, start, end$) // *start is inclusive, end is exclusive.*

2 **If** $end - start \leq 1$ **Then Return**

3 $mid := start + \lfloor (end - start)/2 \rfloor$

4 *MergeSort* ($A, start, mid$)

5 *MergeSort* (A, mid, end)

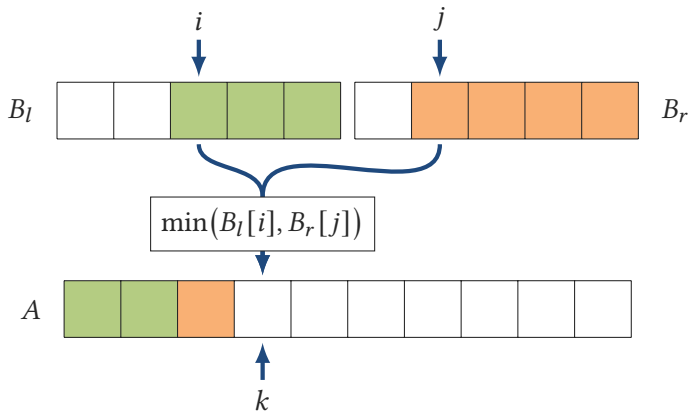
6 Merge ($A, start, mid, end$)

Question: How do we merge?

Merging

Idea

- ▶ Write left and right part into arrays B_l and B_r .
- ▶ Step by step write minimum value from B_l and B_r back into A .



```
1 Procedure Merge (A, start, mid, end)
2   Set  $i := 0$  and  $k := \text{start}$ .
3   Copy  $A[\text{start} : \text{mid}]$  and  $A[\text{mid} : \text{end}]$  into two arrays  $B_l$  and  $B_r$ , respectively.
4   For  $j := 0$  To  $\text{len}(B_r) - 1$  // Copy all entries from  $B_r$ .
5     While  $i < \text{len}(B_l)$  and  $B_l[i] \leq B_r[j]$  // Copy entries smaller than  $B_r[j]$ .
6       Set  $A[k] := B_l[i]$ ,  $i := i + 1$ , and  $k := k + 1$ .
7     Set  $A[k] := B_r[j]$  and  $k := k + 1$ .
8   While  $i < \text{len}(B_l)$  // Copy remaining entries from  $B_l$ .
9     Set  $A[k] := B_l[i]$ ,  $i := i + 1$ , and  $k := k + 1$ .
```

Runtime

- ▶ Each call of *MergeSort* calls itself two times for half the array.
- ▶ Merging costs $\Theta(n)$ time.
- ▶ Thus, $T(n) = 2T\left(\frac{n}{2}\right) + n$, i. e., $T(n) \in \Theta(n \log n)$

Other Properties

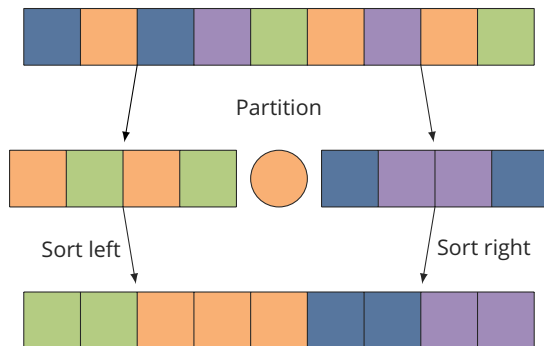
- ▶ Not in-place, $O(n)$ extra memory required
- ▶ Stable (depending on merge implementation)
- ▶ $O(n \log n)$ swaps.

Quicksort

Quicksort

Idea

- ▶ Pick pivot element p .
- ▶ Partition data in two subsets: elements smaller than p and elements larger than p .
- ▶ Sort each part.



Input: An array A of size n and two indices start and end with $0 \leq \text{start} < \text{end} \leq n$.

1 **Procedure** *Quicksort* (A , start , end)

2 **If** $\text{start} < \text{end}$ **Then**

3 $\text{mid} := \text{Partition}(A, \text{start}, \text{end})$

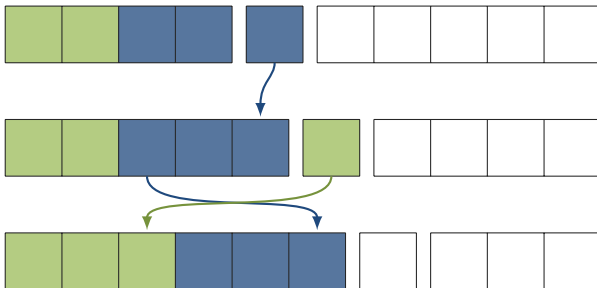
4 $\text{Quicksort}(A, \text{start}, \text{mid})$

5 $\text{Quicksort}(A, \text{mid}, \text{end})$

Partition

Strategy 1

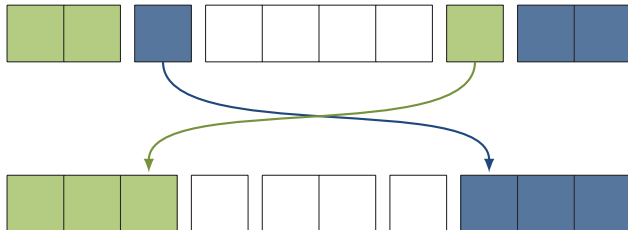
- ▶ Iterate from left to right
- ▶ Grow smaller and larger partition from the left.
- ▶ If element larger than p , nothing to do.
- ▶ If element is smaller than p , exchange with leftmost larger element.



```
1 Procedure Partition ( $A$ , start, end)
2   Pick a pivot-element  $p$  and swap it with  $A[\text{end}]$ .
3   Set  $j := \text{start}$ . // All entries before  $j$  are strictly smaller than  $p$ 
4   For  $i := \text{start}$  To  $\text{end} - 1$ 
5     If  $A[i] < p$  Then
6       Swap  $A[j]$  and  $A[i]$ .
7       Set  $j := j + 1$ .
8   Swap  $A[j]$  and  $A[\text{end}]$ .
9   Return  $j$ 
```

Strategy 2

- ▶ Explore data from both sides.
- ▶ Grow smaller part from the left and larger part from the right.



Strategy 3 (Dutch National Flag Problem)

- ▶ Partition into three groups: smaller, larger, and equal to p .
- ▶ Useful if there are multiple equal elements.
- ▶ Grow smaller/equal part from the left and larger part from the right.



Selecting Pivot Element

Ideal Case

- ▶ p splits data in two equally large parts.
- ▶ Doable in linear time, but constant factor is too large.

Bad Strategy

- ▶ Select first or last element.
(leads to $O(n^2)$ time if already sorted)

Better Strategies

- ▶ Select middle element.
- ▶ Select random element.
- ▶ Select median of three elements.

Runtime

- ▶ Worst case: $O(n^2)$
- ▶ Best case: $O(n \log n)$
- ▶ Average case: $O(n \log n)$
- ▶ Usually faster than other sorting methods (e. g. merge sort).

Other Properties

- ▶ Not in-place, *in average* $O(\log n)$ extra space required (recursive calls)
(if implemented correctly, $O(\log n)$ extra space *in worst case*)
- ▶ Not stable